



Build a Database With Cursor and Supabase MCP



tahirgroot@gmail.com

<https://medium.com/@tahirbalarabe2/how-to-m...>

The screenshot shows a Supabase database table named 'track_likes'. The table has four columns: 'track_id' (text), 'like_count' (int4), 'created_at' (timestamp), and 'updated_at' (timestamp). The table contains 15 rows of data. The background of the screenshot is a colorful, abstract pattern of overlapping circles in yellow, orange, purple, and blue.

track_id	like_count	created_at	updated_at
4JLJ9sR0nUro2s3nNrdx5	20	2026-06-16 02:06:04.931472+00	2026-06-16 02:21:25.634774+00
6dOtVTDdiauQNBQEDOtIAB	2	2026-06-16 02:04:22.682076+00	2026-06-16 02:05:04.968277+00
1BxfuPKGuaTgP7aM0Bbdwr	2	2026-06-16 02:04:17.640851+00	2026-06-16 02:05:04.45351+00
2plbrEY59likOBgBGLjaoe	1	2026-06-16 02:06:14.669701+00	2026-06-16 02:06:14.669701+00
1Qrg8KqIBpW07V7PNxwwwL	1	2026-06-16 02:06:15.656847+00	2026-06-16 02:06:15.656847+00
4ZtFanR9U6ndgddUvNcjcG	1	2026-06-16 02:06:18.230446+00	2026-06-16 02:06:18.230446+00
39LLxExYz6ewLAcYrzQQyP	1	2026-06-16 02:06:19.213082+00	2026-06-16 02:06:19.213082+00
2dHHgzDwk4BjdRwy9uXhTO	1	2026-06-16 02:06:24.566577+00	2026-06-16 02:06:24.566577+00
7qZfU4dYlWlZx7mPBt3	1	2026-06-16 02:06:25.079623+00	2026-06-16 02:06:25.079623+00
0yLdNVWF3Srea0uzk5SzfN	1	2026-06-16 02:06:25.675831+00	2026-06-16 02:06:25.675831+00
3aIlNhkSLSkpJE4MShpDu9	1	2026-06-16 02:06:26.338324+00	2026-06-16 02:06:26.338324+00
4LRPDxqCiklN15c3yImP7	1	2026-06-16 02:06:04.242715+00	2026-06-16 02:06:04.242715+00



Introducing this project!

In this project, I'm going to add a likes feature to NextSound, which is a mini Spotify clone for discovering music. I'll let users heart their favorite tracks and store those like counts in a live Supabase database. I'm doing this project to learn how to use the Model Context Protocol (MCP) inside Cursor. Instead of writing SQL or managing database connections manually, I'll just describe what I want, and Cursor will talk directly to Supabase to set everything up. This teaches me how AI agents can control external services like databases. I'll also build a real portfolio project that works and persists data after a page refresh. No prior database experience needed.

Key tools and concepts

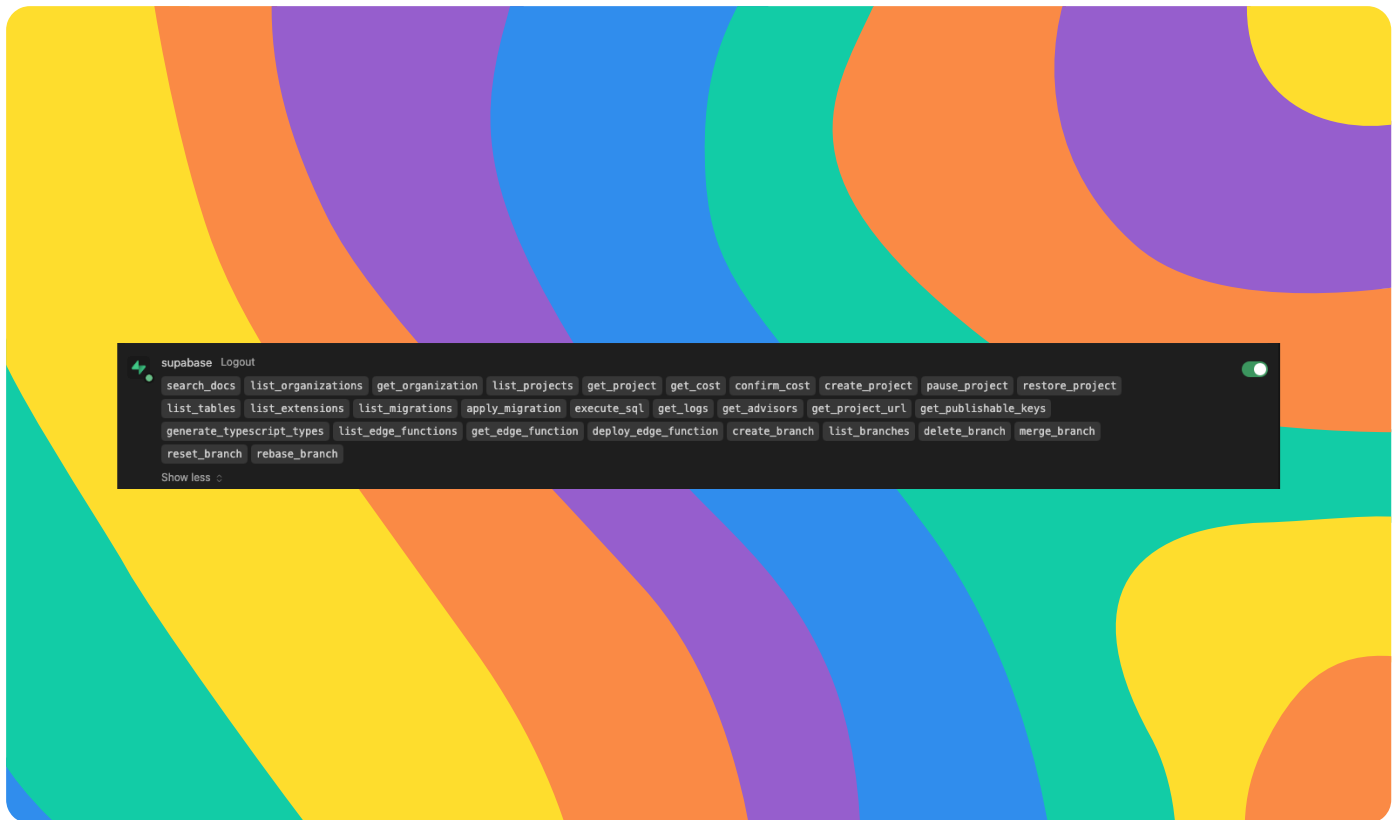
The key tools I used include Cursor with the Supabase MCP, the Supabase dashboard, and the .env file for managing secrets. I also used the `apply_migration`, `execute_sql`, `list_tables`, `get_project_url`, and `get_anon_key` MCP tools. Key concepts I learnt include MCP as a bridge between AI and external services, Row Level Security for data privacy, foreign keys to link tables, and authentication flows with Supabase Auth. The most important thing I learnt was that I can build a full-stack feature—database, UI, and security—using only natural language prompts, without writing SQL or managing backend code manually. This changes how I approach prototyping and app development.

Project completion time

This project took me approximately 2 to 3 hours total, including setup, testing, and the secret mission. The most challenging part was understanding the MCP tool calls and approving each action, especially during the authentication setup with RLS policies. It was most rewarding to see the heart button update the database in real time and then query my own liked tracks from Cursor using plain English. With the skills I learnt, I want to build next a personal playlist manager where users can create custom playlists, add songs, and share them with friends—all powered by Supabase and controlled through Cursor prompts.



Setting up Cursor and MCPs



Why MCPs?

MCP stands for Model Context Protocol, and it helps AI agents like Cursor connect directly to external services such as databases, GitHub, or authentication systems. Without MCPs, AI agents would only chat about code or suggest edits, but they couldn't actually create tables, run queries, or manage a live database. MCP acts like a universal translator: I describe what I want in plain English, and Cursor uses the MCP tools to perform specific actions, like creating a Supabase project or executing SQL. In this step, I installed the Supabase MCP and connected it to my account, so now Cursor can build and manage my database for me.

I connected Cursor to my Supabase account using the Supabase MCP server. I verified this connection by asking Cursor in Agent mode: "What is my Supabase Organization called?" Cursor then used the MCP's `list_organizations` tool, which required my approval to run.



tahirgroot@gmail.com

NextWork Student

nextwork.org

Once I allowed it, Cursor successfully returned the correct organization name I created: "NextWork MCP Project - NextSound Web App." The MCP lets Cursor perform real actions on external services, not just chat. For example, it can now create projects, list tables, or execute SQL directly from the chat, without me writing a single line of database code.



Building with Cursor

How Cursor set up the database

To set up my database, Cursor used a set of tool calls from the Supabase MCP. First, Cursor called `list_projects` to check if I already had any existing projects. Then it used `get_cost` and `confirm_cost` to ensure creating a new project wouldn't incur charges, since Supabase's free tier is fine for this. After confirmation, Cursor called `create_project` to actually build the new Supabase project within my "NextWork MCP Project - NextSound Web App" organization. Cursor then retrieved the `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY` using MCP tools and explained how to put them into a `.env` file. This connected NextSound to its live database.

Understanding `.env` files

A `.env` file stores sensitive information like API keys, database URLs, and authentication tokens. For NextSound, it contains the `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY`, which are the credentials my app needs to connect to the live Supabase database. This is important because these secrets should never be shared publicly or committed to version control like Git. By keeping them in a `.env` file, I protect my database from unauthorized access. Also, Cursor cannot read this file by design, which adds an extra layer of security. Without this file, my web app wouldn't know how to talk to Supabase, and the likes feature wouldn't work.

Implementing the likes feature

The feature I'm building is a likes system for NextSound, where every track card shows a heart button and each track starts with 0 likes. Instead of manually setting up the database, I'm prompting Cursor to read my existing `.env` file for the Supabase credentials, then use the Supabase MCP to create a new table that stores like counts for each track. Cursor will also update the frontend code to display the heart button, handle click events, and sync with the database.

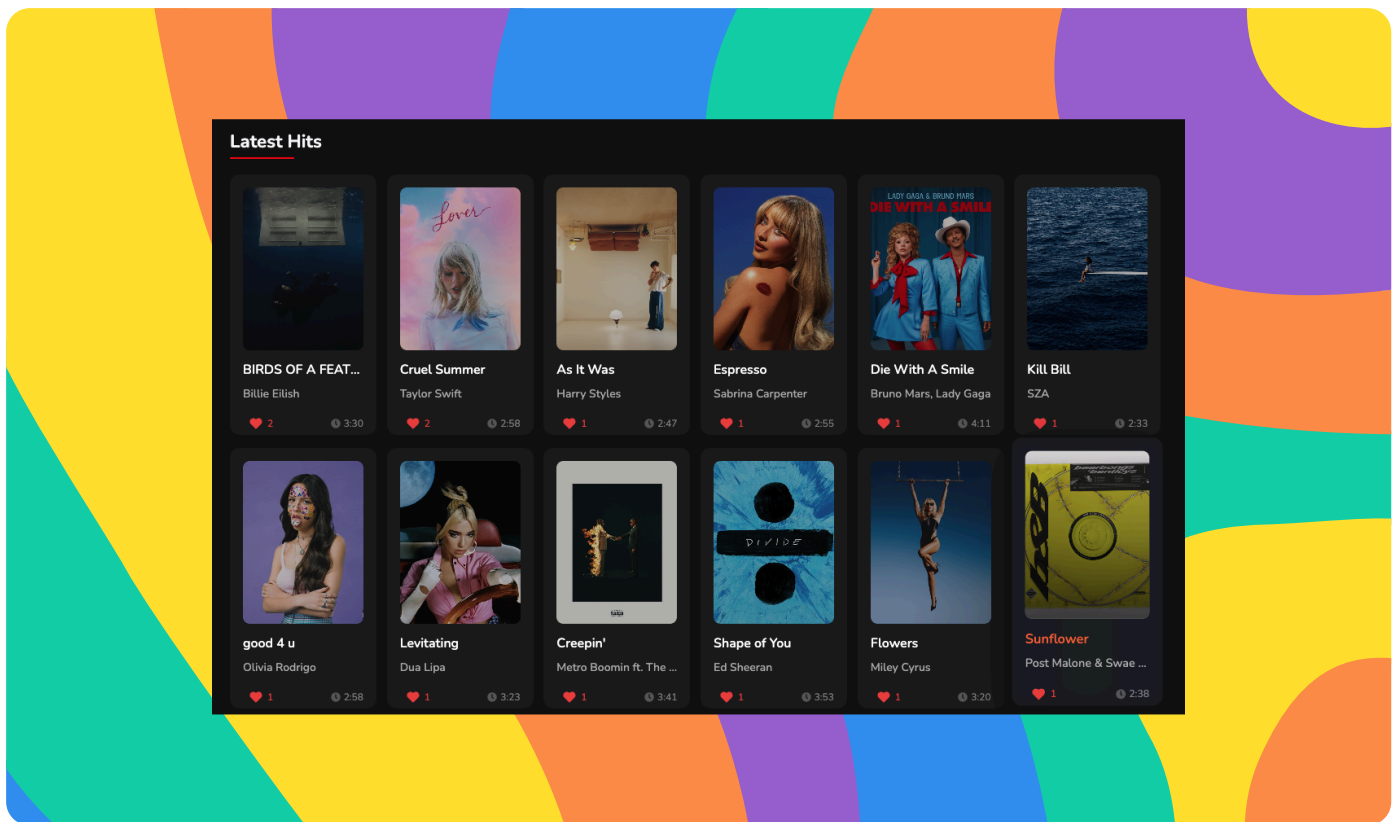
A pink circular logo with the letters 'TA' in white.

This approach saves me from writing SQL or managing API calls by hand, and it ensures the database and UI work together seamlessly from the start.

Cursor's plan for implementing the feature is to first create a database table to store track likes, then initialize all existing tracks with zero likes, and finally update the frontend code to display the heart button and handle clicks. Cursor used tools like `apply_migration` to generate and run the SQL script for creating the `track_likes` table, which includes columns for `track_id` and `like_count`. Then it likely used `execute_sql` to insert initial zero-like records for each track from my music library. After that, Cursor modified the React components to fetch like counts on load, show a heart icon, and send updates to Supabase when clicked. Everything happened from one prompt.



Testing the Likes Feature



How likes connect to the database

When I clicked the heart icon, I could see the like count increase from 0 to 1 and the heart turn red. Clicking again dropped it back to 0 and removed the color. Refreshing the page kept the counts, so the data persisted. Behind the scenes, the database is storing each track's like count in the `track_likes` table. Every click sends an update to Supabase, which changes the number and returns the new value to the app. The heart's color change happens instantly in the UI, but the actual source of truth is the cloud database. This confirms that the frontend, the API calls, and the Supabase table are all working together correctly.

Data in the `track_likes` table

In Supabase, the `track_likes` table stores each track's like count. Each row represents a single track, with columns like `track_id` (a unique identifier for the song) and `like_count` (the number of likes). I verified that the web app's connection with my database works by clicking the heart button in NextSound,



The like_count value in the database matched exactly what I saw in the app. For example, when I liked “Cruel Summer” and the count showed 1, the database row for that track_id also showed 1. This confirms the frontend and backend are perfectly synced and data persists after refresh.

One limitation I notice is that the current track_likes table stores only a global like count for each track, not per user. That means if I like a song, it increases the same counter for everyone. After this project, I would like to improve my database by adding user authentication and a separate user_likes table that tracks which specific user liked which track. Then I could show personalized likes, prevent duplicate likes from the same person, and allow users to see their own liked songs in a playlist. The Supabase MCP can help me add this with another prompt, no manual SQL needed.



Querying with Natural Language

MCP tools used

When I asked Cursor “What tracks are currently liked in NextSound? Use the Supabase MCP,” Cursor used the Supabase MCP to first call `list_tables` to understand the database structure, then `execute_sql` to run a query joining the `track_likes` table with the tracks data. The final answer it gave me was a formatted list of track titles, artists, and their like counts, for example “Cruel Summer by Taylor Swift has 1 like.” This matched exactly what I saw in the web app and the Supabase dashboard. I didn’t write any SQL manually, just described what I wanted in plain English, and Cursor translated that into the right database actions.



Updating Votes with Cursor

track_id	like_count	created_at	updated_at
4jLjU9sR0nUro2s3nNrdx5	20	2026-06-16 02:06:04.931472+00	2026-06-16 02:21:25.634774+00
6dOtVTDdiaoQNBQEDOtIAB	2	2026-06-16 02:04:22.682076+00	2026-06-16 02:05:04.968277+00
1BxfuPKGuaTgP7aM0Bbdwr	2	2026-06-16 02:04:17.640851+00	2026-06-16 02:05:04.45351+00
2plbrEY59likOBgBGLjaoe	1	2026-06-16 02:06:14.669701+00	2026-06-16 02:06:14.669701+00
1Qrg8KqjBpW07V7PNxwwwL	1	2026-06-16 02:06:15.656847+00	2026-06-16 02:06:15.656847+00
4ZIFanR9U6ndgddUvNcjG	1	2026-06-16 02:06:18.230446+00	2026-06-16 02:06:18.230446+00
39LLxEXyZ6ewLAcYrzQQyP	1	2026-06-16 02:06:19.213082+00	2026-06-16 02:06:19.213082+00
2dHHgzDwk4BJdRwy9uXhTO	1	2026-06-16 02:06:24.566577+00	2026-06-16 02:06:24.566577+00
7qiZIU4dYIiWlZx7mPBli3	1	2026-06-16 02:06:25.079623+00	2026-06-16 02:06:25.079623+00
OyLdNVWF3Srea0uzk55zFn	1	2026-06-16 02:06:25.675831+00	2026-06-16 02:06:25.675831+00
3a1INhkSLSkpJE4MSHpDu9	1	2026-06-16 02:06:26.338324+00	2026-06-16 02:06:26.338324+00
4LRFXqCikLIN15c3ylmP7	1	2026-06-16 02:06:04.242715+00	2026-06-16 02:06:04.242715+00

How Cursor updated the votes

Aside from querying the database, I also used Cursor to update the like count for a specific track. I asked Cursor to set the votes for “Espresso” to 20. Cursor first searched my project files, likely `src/data/tracks.ts`, to find the internal `track_id` for that song. Then it used the Supabase MCP to run an `execute_sql` tool with an `UPDATE` command targeting that specific row in the `track_likes` table. I verified the update by refreshing the NextSound web app, where “Espresso” showed 20 likes, and by checking the Supabase Table Editor, where the matching row also had 20. This shows I can modify live database records using plain English from inside Cursor.

Verifying the update

Being able to update data with natural language is powerful because it removes the need to remember SQL syntax or navigate complex database tools. I could use this to quickly fix incorrect like counts, seed new tracks with default values,



tahirgroot@gmail.com

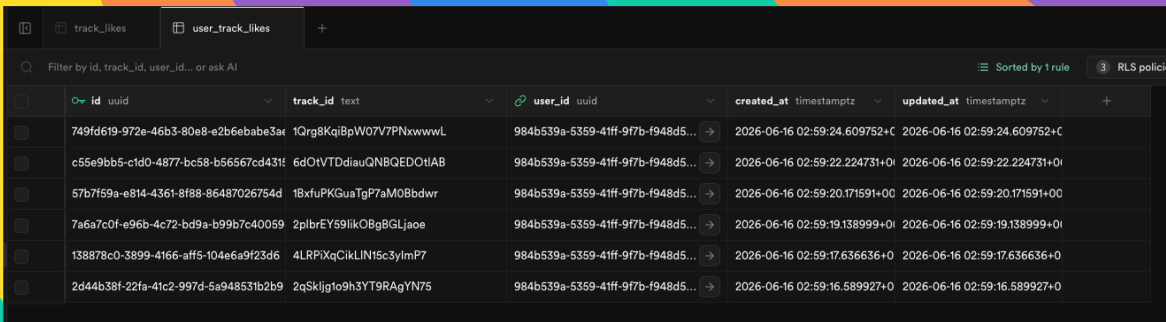
NextWork Student

nextwork.org

For example, if a manager asks “how many songs have over 100 likes?”, I can just ask Cursor and get an answer instantly. It also helps me prototype features faster, like resetting all likes to zero for a fresh start, or copying popular tracks to a playlist table. This makes database work feel conversational and accessible.



Secret Mission: User Authentication



id	user_id	track_id	created_at	updated_at
749fd619-972e-46b3-80e8-e2b6ebabe3ae	1Qrg8KqiBpW07V7PNxwwwL	984b539a-5359-41ff-9f7b-f948d5...	2026-06-16 02:59:24.609752+00	2026-06-16 02:59:24.609752+00
c55e9bb5-c1d0-4877-bc58-b56567cd43f1	6dOtVTDdiauQNBQEDotIAB	984b539a-5359-41ff-9f7b-f948d5...	2026-06-16 02:59:22.224731+00	2026-06-16 02:59:22.224731+00
57b7f59a-e814-4361-8f88-86487026754d	1BxfuPKGuaTgP7aM0Bbdwr	984b539a-5359-41ff-9f7b-f948d5...	2026-06-16 02:59:20.171591+00	2026-06-16 02:59:20.171591+00
7a6a7c0f-e96b-4c72-bd9a-b99b7c40059	2plbrEY59likOBg8GLjaoe	984b539a-5359-41ff-9f7b-f948d5...	2026-06-16 02:59:19.138999+00	2026-06-16 02:59:19.138999+00
138878c0-3899-4166-aff5-104e6a9f23d6	4LRPiXqCikLINI5c3ylmP7	984b539a-5359-41ff-9f7b-f948d5...	2026-06-16 02:59:17.636636+00	2026-06-16 02:59:17.636636+00
2d44b38f-22fa-41c2-997d-5a948531b2b9	2qSkIjgto9h3YT9RAgYN75	984b539a-5359-41ff-9f7b-f948d5...	2026-06-16 02:59:16.589927+00	2026-06-16 02:59:16.589927+00

user_profiles vs user_track_likes

The `user_profiles` table stores user account information, such as `user_id`, email, and `display_name`, linking each Supabase Auth user to their profile data. The `user_track_likes` table stores the relationships between users and tracks: each row contains a `user_id` and a `track_id`, which together record that a specific user has liked a specific track. This table has no global like count; instead, likes are personal. By using foreign keys, Supabase connects these tables so I can query all tracks liked by a given user or all users who liked a specific track, enabling personalized features and Row Level Security.

Linking users to tracks

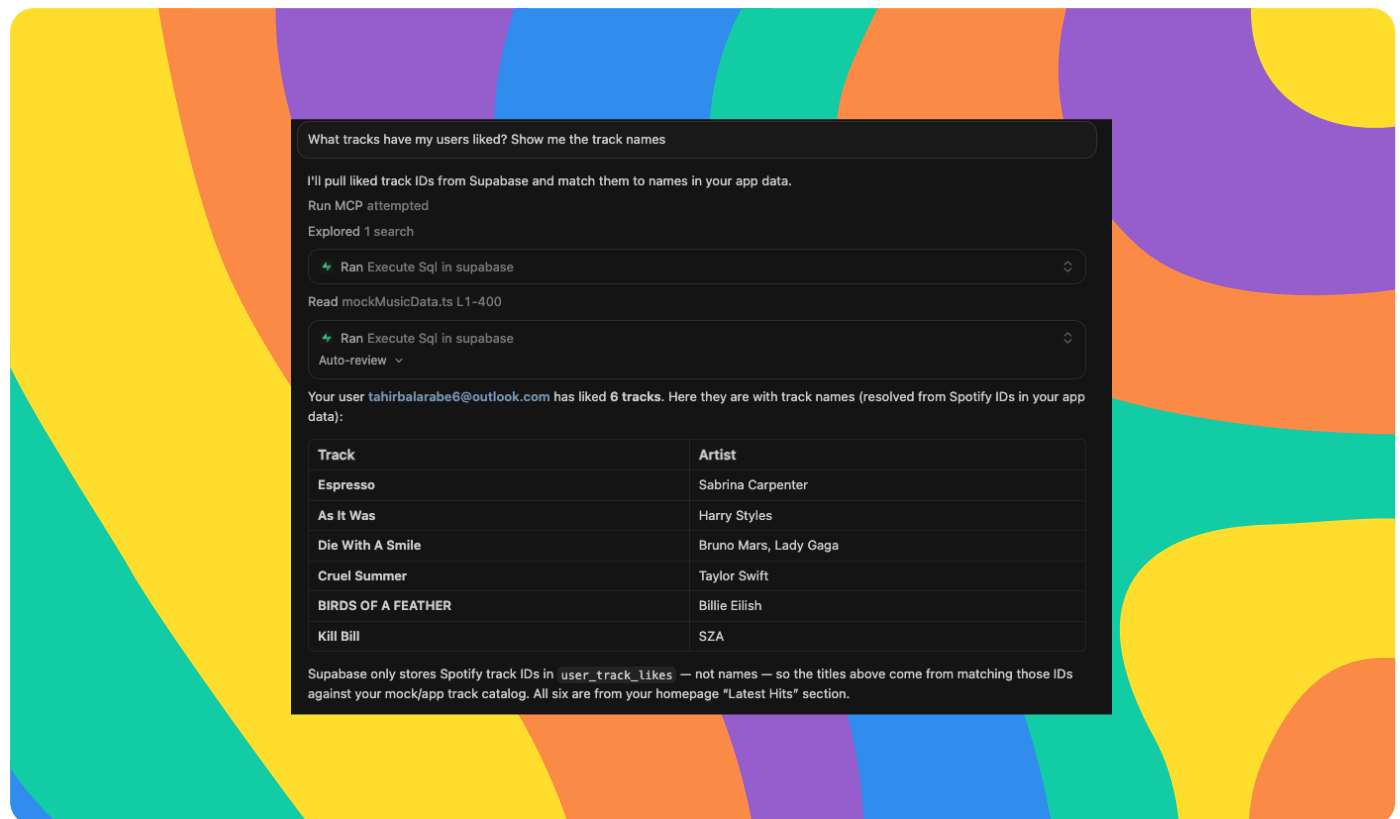
The database matches a user to the tracks they've liked by matching the `user_id` column in the `user_track_likes` table to the `id` column in the Supabase Auth users table, which is also linked to `user_profiles`.



When I click the heart button while logged in, the app sends my authenticated `user_id` along with the `track_id` to Supabase, and a new row is inserted with both values. This makes it possible to know exactly which tracks a specific user has liked, and also to count total likes for a track by counting rows with that `track_id`. Row Level Security ensures I can only see and modify my own rows, keeping everyone's likes private.



Querying User Data



Benefits of natural language querying

Storing user emails means holding personally identifiable information, which requires careful handling to prevent leaks or unauthorized access. One way to minimize risk is to use Supabase's built-in Row Level Security to restrict access so only the authenticated user or admins can view their own email. Another is to avoid logging or exposing emails in frontend code or error messages.

Supabase's free tier allows up to 50,000 monthly active users, but be aware of limitations such as rate limits on authentication requests and the need to enable email confirmations to prevent fake sign-ups. Also, always use HTTPS and consider additional verification like CAPTCHA on sign-up forms.



Other questions to explore

I could also ask Cursor questions like “Which track has the most total likes across all users?” or “Show me users who haven’t liked any tracks yet.” I could ask “What are the top 5 most liked songs this week?” or “List all tracks liked by user with email example@email.com.” I could also request analytics like “How many likes did each artist receive in total?” or “What’s the average number of likes per user?” For debugging, I might ask “Are there any duplicate like records for the same user and track?” or “Which user liked the most tracks?” Finally, I could combine data: “Show me each user’s email and the count of tracks they liked.”



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

